

AI dev environment setup

- 1.1 Enabling AI debug tools
- 1.2 AI Debug menu
- 1.3 AI constants
- 2. Shared XS setup
- 3. VS Code open folder
- 4.1 AI configs
- 4.2 Normal config setup:
- 4.3 Iteration speedup configs
- 5. Important AI hotkeys

1.1 Enabling AI debug tools

For this step to be possible we need to first understand hotkeys + configs and have the related files set up. Read and follow “*BANG documentation\Config documentation\Config basics*” and “*BANG documentation\Hotkey documentation\Hotkey basics*”.

You should now have a *user.con* and *user.cfg* and understand how to use them.

The one **essential** config we need to enable is *aiDebug*. This config by itself will enable all actual debug tools for us. There are more AI related configs, which can be found lower on this page, but they’re situational.

To verify our *aiDebug* config was correctly enabled and our hotkeys work we will run through all the debug tools while in-game:

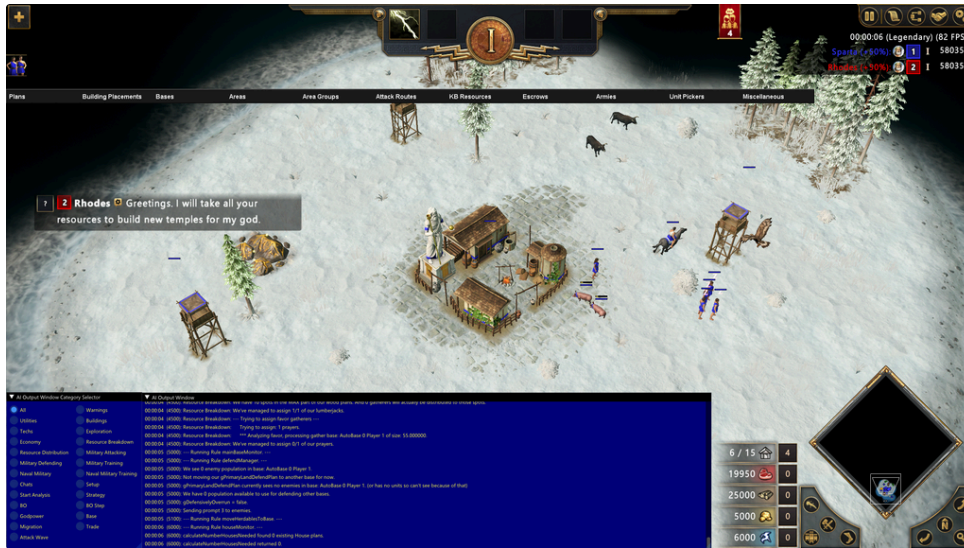
1. Restart your game to have the *aiDebug* config take effect.
2. In the skirmish lobby set yourself as an AI.
3. Start the game.

1.2 AI Debug menu

 Verify we have this menu available to us.

Press “alt-shift-q” (or your own bind if you rebound it).

You should now see the menu come up, if it doesn’t your *aiDebug* config wasn’t properly set.



The AI Debug menu is the core of all the available AI tools. By itself it has all the information from AI/KB related systems that we need. Many other AI configs actually modify what is displayed on this menu. For example, `aiOutputPlanAssignment` puts all the text it generates into the AI Debug output window.

1.3 AI constants

i Verify we have these files available to us.

Start any game that has AI in it, which you presumably already did to verify the `aiDebug` Menu / XS Debugger.

Navigate to: `"...\\Users|<your username>|Games\\Age of Mythology Retold\\temp\\Logs"`

Naming style: `MythAIConstantsPlayer1.txt`

If you don't see the text files present your `aiDebug` config wasn't properly set.

Many AI/KB syscalls want an ID as input instead of a name. To lookup what IDs map to what entity in the game we have the AI constants file. These constants act like what in coding is known as an "enum". Example usage:

```
kbUnitCount(cUnitTypeHillFort, cMyID, cUnitStateAlive)
```

In the example you can clearly read that we're trying to count Hill Forts. But `cUnitTypeHillFort` actually stands for 347 and that is also what you get if you echo that constant. We turn that number into a name because otherwise we have no idea what we're reading as humans. For example:

```
kbUnitCount(87, cMyID, 4)
```

The line above is counting Minotaurs that are dead, but how would you know that at a glance? This is why using the constants over actual numbers is always desirable. Maybe you also already noticed I changed `cUnitStateAlive` to a number, because that variable in fact is also a constant/enum.

End of in-game setup part, we have visited all *aiDebug* related tools and verified they work.

2. Shared XS setup

Read the `"BANG documentation\\XS documentation\\How-to Doxygen"` document to get a better understanding of all possible syscalls.

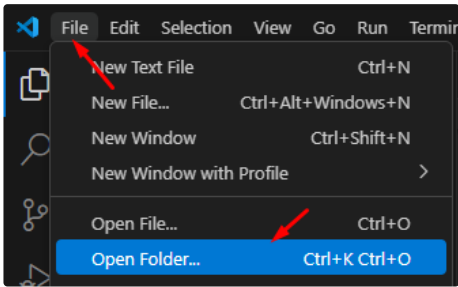
Follow the steps in: `"BANG documentation\\XS documentation\\VS Code Extension For XS\\Installing VS Code and extension for XS"`.

Don't forget to also follow the steps in: *"BANG documentation\XS documentation\VS Code Extension For XS\VS Code adjust settings"*.

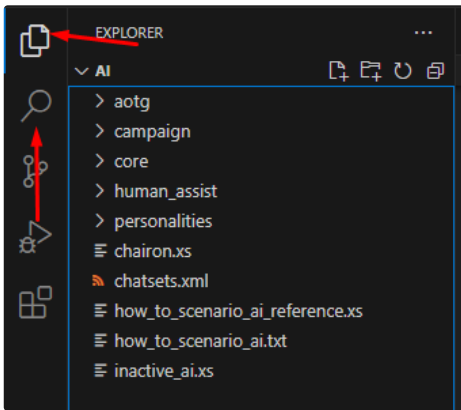
Read up on the XS Debugger in: *"BANG documentation\XS documentation\XS debugger"*.
When the *aiDebugScript* config is enabled the XS debugger will automatically breakpoint when you hit a warning.

3. VS Code open folder

When working on a set of AI files you can open any containing folder like this:



When you have a folder open you can see + search in all the files inside that folder inside VS Code. This is of great help when you have multiple relevant files.



4.1 AI configs

This is a table of all AI related configs:

AlvsAI	Replaces yourself, player 1, by an AI. This can also be achieved inside the skirmish lobby screen.
aiDebug (IMPORTANT)	Enables aiDebug window, Enables XSDebugger, generates AI constants, enables all echos, writes all echo output to a file at game's end.
aiDebugScript	AI Warnings cause the XS debugger to breakpoint.
aiBreakAtStart	Breakpoints the XS Debugger at the start of main().

aiRecompileScripts	Recompiles AI scripts when loading savegames.
aiHideWarningEchoes	Warnings from the source + aiEchoWarning won't be displayed.
aiHideEngineInfoEchoes	Info echoes from the engine won't be displayed.
aiOutputPlanDestructionAndCreation	Logs whenever the destroy flag is set on a plan or when a new plan is created.
aiOutputResourceSelection	Displays debug output about the selecting of a new KB Resource by the gather plan in the "AI Debug Output" window.
aiOutputUnitAssignment	Spews debug output about the plan assignment in the "AI Debug Output" window.
aiOutputFindBestUnit	Spews debug output about the selecting of units for plans in the "AI Debug Output" window.
aiOutputPlanAssignment	Spews debug output about the actual adding/removing/loaning units from plans in the "AI Debug Output" window.
aiChatDelay	Delay between valid AI chats, defaults to 180,000ms
OutputSlowRuleWarnings	In the aiDebug's output window under the warning section you will be notified if a rule took longer than 5ms to complete, which would mean it's lagging the game more than we want.

The following configs can be defined in the *user.cfg* but can also be toggled in-game via the aiDebug's Misc menu.

aiGraphicalBaseDebug	Turns on rendering of base radius and threat level.
aiGraphicalPlanDebug	Turns on rendering of plan radius and type for the opened plans.
aiGraphicalPlanDebugAll	Turns on rendering of plan radius and type for all plans.
aiShowBPValueText	Shows the Building placement values over each position.
aiShowAreaDangerRatings	Shows the area danger ratings.

4.2 Normal config setup:

As you can see above there are a ton of AI related configs. But you don't want all of them enabled all the time. Some of them create extensive logging in the aiDebug output window that is not desired if you're not looking for something specific. By default you should be running: **AlvsAI and aiDebug**. You can have **aiDebugScript** on at all times to catch errors you make, but it can also be annoying to have the XS debugger breakpoint when you're working on something else.

(You can skip **AlvsAI** if you just change yourself to an AI in the lobby)

4.3 Iteration speedup configs

This is a table of all iteration speedup related configs (not AI specific):

noIntroCinematics	Skips all videos displayed during game startup
disableAssetPreloading	Disables asset preloading when starting a game. Since you will be potentially reloading a map many times you don't want to wait on this, you can use this for shorter load times.
runAsFastAsPossible	Tries to run the game faster than normal, to test AI logic quicker. How much faster the game runs is capped by your CPU speed.

5. Important AI hotkeys

If you wish to not use the default binds then copy the lines in the left column to the *user.con* and change "somekey" to your wanted binding. You can look in *game.con* for examples of possible bindings.

map("somekey", "aidebug", "AIDebugInfoToggle")	Opens the debug window associated with the aiDebug config. Alt-shift-q is the default binding.
map("somekey", "root", "gadgetToggle("XSDebugger")")	Open the debug window associated with debugging XS, alt-shift-d is the default binding.
map ("somekey", "world", "player(1)")	Switches to player 1 in SP games, control-f1 is the default binding scheme, control-f2 for player 2 etc. Switching to Mother nature has a different default: map ("control-shift-f10", "world", "player(0)")